Table of Contents


```
clc;
clear all;
close all;
```

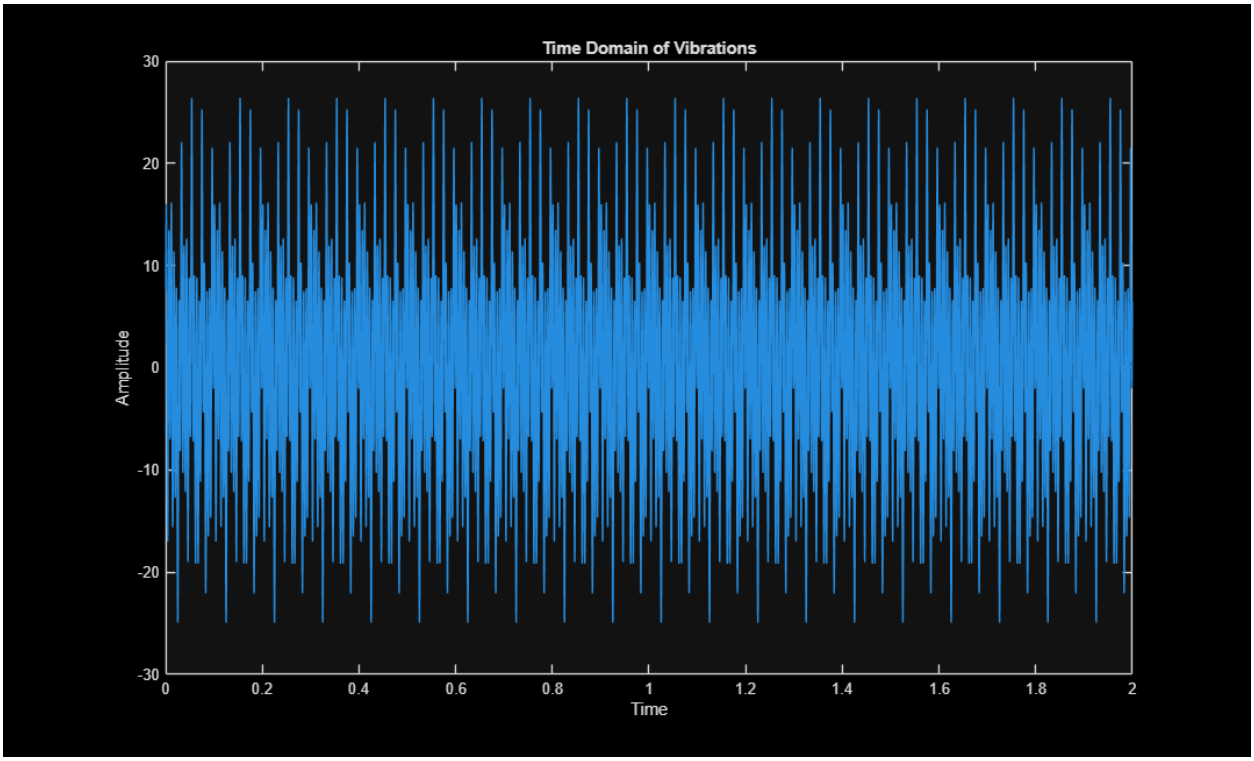
EEL 3135 Lab 4

Name:Evan Baesler Date: 09-24-2025

```
% Answer ALL questions in comments

%{
Example of a
block comment, might be
useful instead of commenting every line.
%}
```

Part 1 Working within the Time Domain

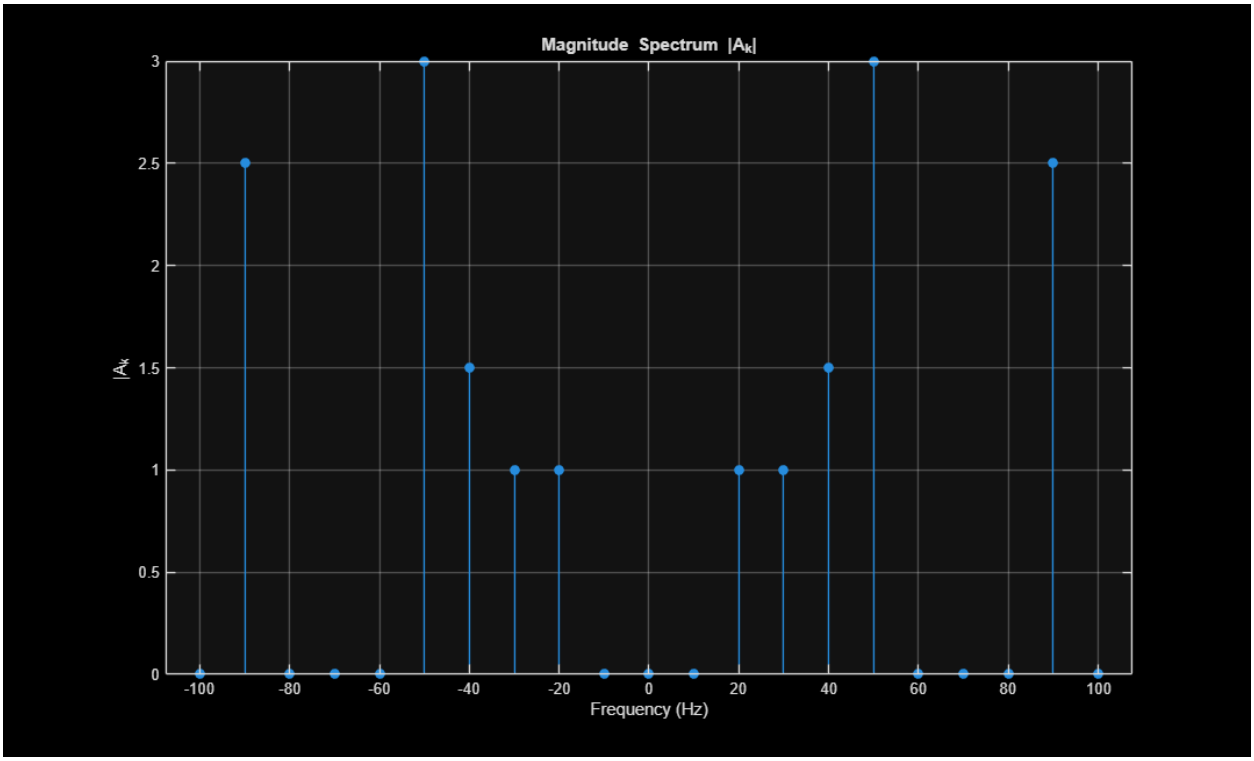
```
% Import your waveform and time data
myUFID = 31151619;
[Y_t, T_s] = BuildingTraceData(myUFID);

% Question 1
figure
plot(T_s, Y_t)
title('Time Domain of Vibrations')
ylabel('Amplitude')
xlabel('Time')

% Question 2

% Yes the signal is periodic with a T of 0.100s

% Question 3

T_0 = 0.1; % Fundamental Period (must fill in answer here)

% Question 4

f_0 = 10; % Fundamental frequency (must fill in answer here)
T_0 = 0.1; % Fundamental Period (if needed to recalculate)
```

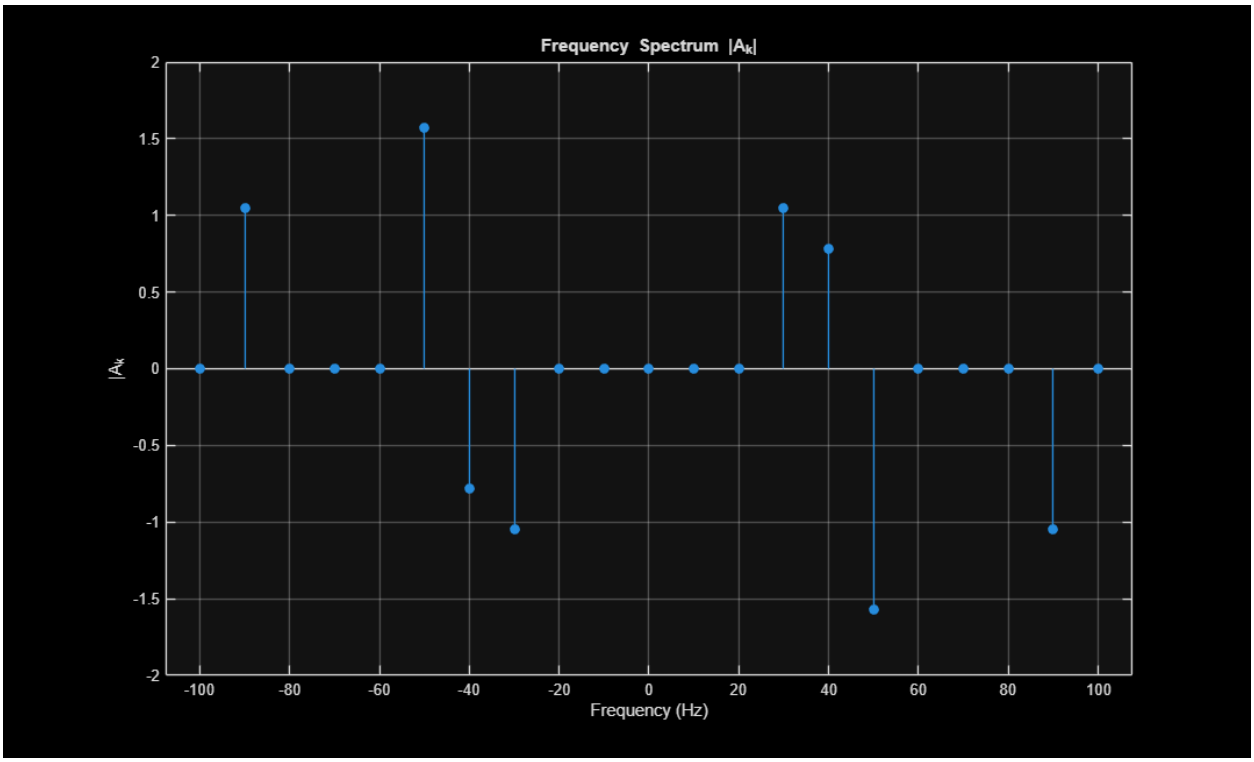


Part 2 Fourier Series (Analysis)

```
% Question 5

% The below code is given as a starting point.
K = 10;
% Find Index point within Ts for the first T_o
[~,L_T_o]=min(abs(T_0-T_s)); % help min, help abs MIGHT be useful!

%Pre-Allocation of space for Matrices  (speeds up Matlab)
Kernel_R = zeros(2*K+1,L_T_o);
Kernel_I = zeros(2*K+1,L_T_o);

% Calculation of coefficient matrix
% trapz was used as a way to approximate the integral
% help trapz may be useful to type into your command window

n=1;
for  k = -K:K
     Kernel_R(n,:) = Y_t(1:L_T_o) .* cos(2*pi*k*f_0*T_s(1:L_T_o));
     Kernel_I(n,:) = Y_t(1:L_T_o) .* sin(2*pi*k*f_0*T_s(1:L_T_o));
     A_R_k(n) = (1/T_0)*trapz(T_s(1:L_T_o),Kernel_R(n, :));    % Real
     A_I_k(n) = (1/T_0)*trapz(T_s(1:L_T_o),Kernel_I(n, :));    % Imaginary
     A_k(n)= A_R_k(n)+i*A_I_k(n);
     n=1+n;
end

% Because trapz approximation might cause our conjugate pairs to be
```

```
% slightly off (leading to imaginary values in the time domain),
% we can round off to 3 digits.
A_k = round(A_k,3);

% Question 6
% Hint: For each frequency, there MUST be a negative frequency pair
% for each value's conjugate A_k so A_k and Freqs must be the same size

Freqs = (-K:K) * (1/T_0);

% Question 7
% Hint: help stem might be useful to type in your command window

figure;
stem(Freqs, abs(A_k), 'filled');
title('Magnitude Spectrum |A_k|');
xlabel('Frequency (Hz)');
ylabel('|A_k');
grid on;

% Question 8
% Hint: help stem might be useful to type in your command window

figure;
stem(Freqs, angle(A_k), 'filled');
title('Frequency Spectrum |A_k|');
xlabel('Frequency (Hz)');
ylabel('|A_k');
grid on;

% Question 9

threshold = 1/10000;  % small number incase of rounding from numerical
methods
Freq_nonzero = [];

for n = 1:length(A_k)
    if abs(A_k(n)) > threshold
        Freq_nonzero(end+1) = Freqs(n);
    end
end

% Question 10

% By definition, an nth harmonic is n times faster than the first harmonic.
% This is because in an array of harmonics, the fundamental frequency is
% your step. For every step in the same direction, you are moving n times
% faster than the first harmonic.
```



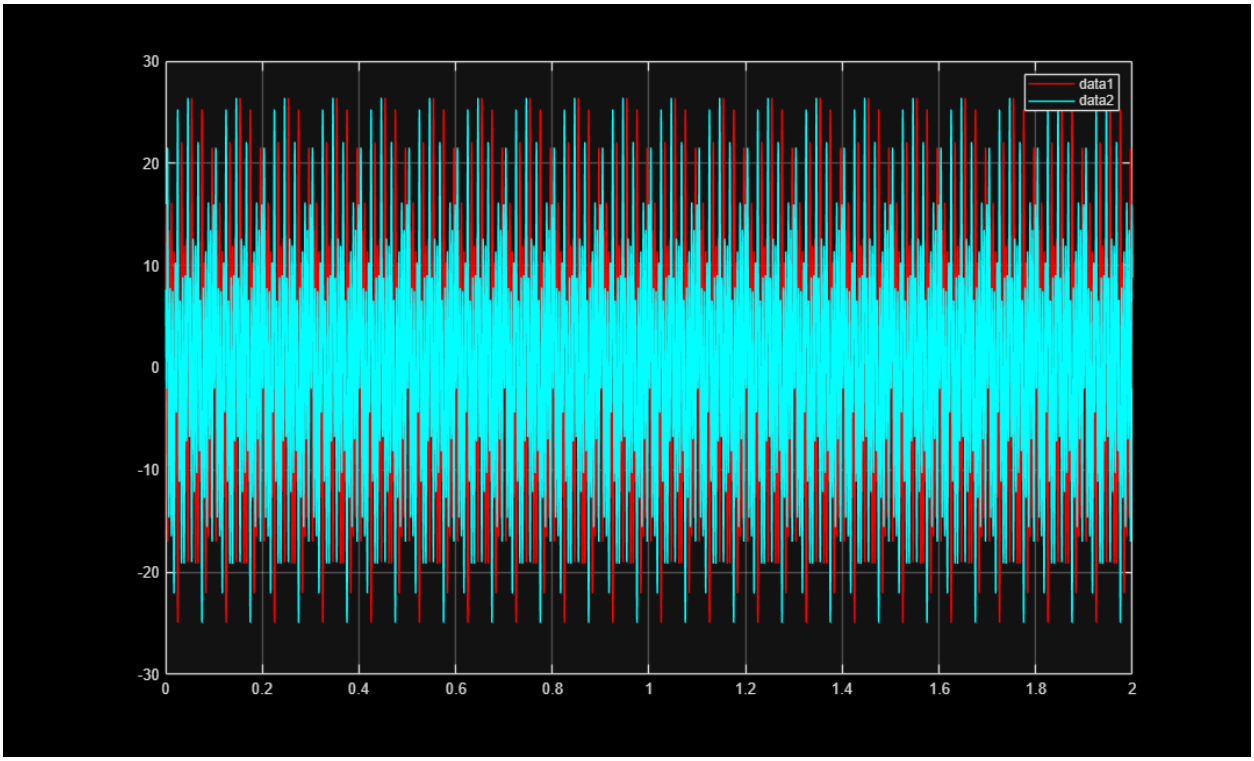


Part 3 Fourier Series (Synthesis)

```
% Question 11
```

```
% Note: Because of rounding approximating errors in Matlab,
% it might say that your reconstructed signal is imaginary even though the
% magnitude of the imaginary componet is in the power of 10^-15.
% Something like max(imag(ReconstructedSignal)) could be used to test if
% your signal has this problem, if so, use real(ReconstructedSignal) to
% remove those small imaginary rounding errors.
% WARNING: IF max(imag(ReconstructedSignal)) RETURNS A VALUE LARGER THAN
% 10^-10,YOUR CODE IS NOT CORRECT!

ReconstructedSignal = zeros(size(T_s));

n = 1;

for k = -K:K

     ReconstructedSignal = ReconstructedSignal + A_k(n) *
exp(1i*2*pi*k*f_0*T_s);
     n = n+1;

end

max(imag(ReconstructedSignal)); % returns 2.2204e-15

ReconstructedSignal = real(ReconstructedSignal);

% Question 12
% Hint: help hold might be useful to type in your command window
% Exploring the matlab plot legend and color options for plots
% might make it easier to read. Be sure to explain what your data.

plot(T_s,Y_t,'r')% T_s, ReconstructedSignal, 'b--');
hold on;
grid on;

% Question 13

% Improved attempt at analysis goes here
% Hint: What could we do to better analyze the signal?

% How I will improve: Lets remove rounding and increase harmonics captured.

% Improved attempt at synthesis goes here

K = 30;

Kernel_R = zeros(2*K+1,L_T_o);
Kernel_I = zeros(2*K+1,L_T_o);

n=1;
for  k = -K:K
     Kernel_R(n,:) = Y_t(1:L_T_o) .* cos(2*pi*k*f_0*T_s(1:L_T_o));
     Kernel_I(n,:) = Y_t(1:L_T_o) .* sin(2*pi*k*f_0*T_s(1:L_T_o));
     A_R_k(n) = (1/T_0)*trapz(T_s(1:L_T_o),Kernel_R(n, :));   % Real
     A_I_k(n) = (1/T_0)*trapz(T_s(1:L_T_o),Kernel_I(n, :));   % Imaginary
```

```
     A_k(n)= A_R_k(n)+i*A_I_k(n);
     n=1+n;
end

Freqs = (-K:K) * (1/T_0);

threshold = 1/10000;  % small number incase of rounding from numerical
methods
Freq_nonzero = [];

for n = 1:length(A_k)
    if abs(A_k(n)) > threshold
        Freq_nonzero(end+1) = Freqs(n);
    end
end

ReconstructedSignal = zeros(size(T_s));

n = 1;

for k = -K:K

     ReconstructedSignal = ReconstructedSignal + A_k(n) *
exp(1i*2*pi*k*f_0*T_s); % DC
     n = n+1;

end

max(imag(ReconstructedSignal)); % returns 2.2204e-15

ReconstructedSignal = real(ReconstructedSignal);

plot(T_s, ReconstructedSignal, 'c')
legend;



% Question 14
% For full points, provide more than a single sentence.

% To get better accuracy, I used more than the 10 origianlly provided
% harmonics. This allowed me to make sure my amplitudes matched better by
% taking the contributions from latter harmonics, even if they do not add
% much, the sum of their contributions can add up.
```



Published with MATLAB® R2025a